SpecFirst Phase 3

Seven-Layer Rulebook / Manifest Loading Architecture

Purpose: Phase 3 loads and validates the trusted local rulebook selected by Phase 2. It is the manifest trust gate: no Bob calls, no patching, no test generation, and no accessibility compliance claims.

1. Phase 3 in the pipeline

Phase 3 sits between pattern classification and spec generation. Phase 2 decides the UI pattern and whether the pipeline may continue. Phase 3 loads the local rulebook for that pattern and validates that it is safe, versioned, source-backed, and complete enough for Phase 4.

```
Phase 1: componentAnalysis.json
-> Phase 2: classification.json
-> Phase 3: registry lookup + seven-layer manifest validation
-> Phase 4: component-specific locked spec
```

2. Input and output

Phase 3 consumes the Phase 2 artifact, not the raw source file. It should trust only the explicit handoff in classification.json.

Item	Path / value	Purpose
Input	specfirst/runs/<run-id>/classification.json	Phase 2 decision, confidence, status, nextPhase.canContinue, and manifestId.
Registry	specfirst/rules/registry.json	Allowed manifest set. Maps manifest IDs to local versioned files and support status.
Manifest	specfirst/rules/react-select-only-combobox.v1.json	Seven-layer rulebook for the supported MVP pattern.
Output	specfirst/runs/<run-id>/manifestLoadResult.json	Records loaded manifest, validation results, source layers, and Phase 4 handoff.
Snapshot	specfirst/runs/<run-id>/manifestUsed.json	Exact manifest copy used in this run for auditability.

3. What Phase 3 must do

Phase 3 should run as a strict sequence of validation gates. Every gate either passes or stops the pipeline with a clear reason.

Gate	Question answered	Failure behavior
Continuation gate	Did Phase 2 set canContinue=true?	Stop and write a blocked manifestLoadResult.
Manifest ID gate	Is nextPhase.manifestId present?	Stop. Do not guess a manifest.
Registry gate	Is manifestId present in registry.json?	Stop. Pattern unsupported or registry misconfigured.
File gate	Does the manifest file exist?	Stop. Missing rulebook.
Schema gate	Does the manifest validate against manifest.schema.json?	Stop. Manifest malformed.
Pattern gate	Does manifest.id match classification/manifestId?	Stop. Avoid wrong rules.

Gate	Question answered	Failure behavior
Seven-layer gate	Does manifest include the required source-layer sections?	Stop or warn depending on severity.
Boundary gate	Does manifest define manual review and report boundary language?	Stop. Avoid compliance overclaim.
Applicability gate	Does manifest applicability agree with Phase 2?	Stop if violated.

4. The seven-layer rulebook model

The manifest should not be a flat checklist. It should be a versioned rulebook built from seven distinct layers. These layers make the rules traceable, testable, conservative, and defensible.

1. WCAG outcome layer
Maps requirements to high-level accessibility outcomes such as keyboard operation, no keyboard trap, and name/role/value.

2. WAI-ARIA APG pattern layer
Defines expected select-only combobox behavior: trigger, popup, listbox, options, expanded state, and keyboard model.

3. ARIA validity layer
Constrains roles, states, and properties so Bob does not add invalid or inappropriate ARIA.

4. Accessible-name layer
Defines allowed naming strategies and role/name query expectations.

5. Automation layer
Maps requirements to Playwright and axe-core test strategies.

6. Manual review boundary layer
Lists what automated tests cannot honestly verify.

7. SpecFirst safety policy layer
Defines reject-if rules, native-first policy, Bob constraints, and forbidden report claims.

5. Layer details

Layer 1: WCAG outcome layer

Explains why a rule exists. It maps requirements to outcomes such as WCAG 2.1.1 Keyboard, 2.1.2 No Keyboard Trap, 1.3.1 Info and Relationships, and 4.1.2 Name, Role, Value. WCAG is an outcome mapping layer, not an operational checklist.

Layer 2: WAI-ARIA APG pattern layer

Describes the expected behavior for the supported pattern. For a select-only combobox, it covers trigger semantics, popup association, listbox/option semantics, expanded state, selected state, and keyboard interactions such as ArrowDown, Enter, and Escape.

Layer 3: ARIA validity layer

Prevents invalid or inappropriate ARIA patches. It should define allowed host elements, required attributes, allowed popup roles, and constraints for option states. For MVP, this can be a lightweight rule table, not a full ARIA validator.

Layer 4: Accessible-name layer

Defines how a component can satisfy the accessible-name requirement. Allowed strategies can include aria-label, aria-labelledby, or visible label association. The test strategy should usually be a Playwright role/name query.

Layer 5: Automation layer

Defines which rules become automated tests. It maps requirements to strategies such as role-query, attribute-toggle, keyboard-open, keyboard-select, keyboard-close, and axe-scan.

Layer 6: Manual review boundary layer

Explicitly lists what remains manual: screen reader announcement quality, visual focus quality, whether native select is preferable, cross-browser assistive technology behavior, and whether option labels make sense in product context.

Layer 7: SpecFirst safety policy layer

Protects the pipeline from unsafe remediation. It includes rejectIf criteria, native-first policy, Bob patch constraints, and report wording boundaries such as forbidding “WCAG 2.2 AA compliant.”

6. Recommended manifest structure

The manifest should be stored locally and versioned. For the MVP, the only fully supported manifest is react-select-only-combobox.v1.json. It should be detailed enough for Phase 4 and Phase 5 to generate a locked spec and tests, but conservative enough to avoid overclaiming.

```
{
"id": "react-select-only-combobox",
"version": "1.0.0",
"displayName": "React Select-Only Combobox",
"sourceModel": "7-layer",
"sources": {
"wcagOutcomes": [],
"ariaPattern": [],
"ariaValidity": [],
"accessibleName": [],
"automation": [],
"manualReview": [],
"specfirstPolicy": []
},
"applicability": {
"requires": [],
"rejectIf": []
},
"requirements": [],
"automation": {},
"manualReviewRequired": [],
"bobPatchConstraints": [],
"reportBoundary": {}
}
```

7. Requirement schema

Each requirement should be traceable across layers. This lets the team explain where a rule came from, how it will be tested, and what remains outside automated coverage.

```
{
"id": "expanded-state",
"category": "state",
"description": "The combobox exposes whether its popup is expanded or
collapsed.",
"layers": {
"wcagOutcomes": ["4.1.2"],
"ariaPattern": ["combobox-expanded-state"],
"ariaValidity": ["aria-expanded-allowed-on-combobox-control"],
"automation": ["attribute-toggle-test"]
},
"testable": true,
"maturity": "automated",
```

```
"testStrategy": "attribute-toggle",
"manualReview": false
}
```

8. MVP rule set for react-select-only-combobox

- **accessible-name:** Combobox exposes a non-empty accessible name. Automated via role/name query.
- **combobox-role:** Primary control exposes combobox semantics. Automated via role query.
- **expanded-state:** aria-expanded reflects popup visibility. Automated via attribute toggle.
- **popup-associated:** Combobox is associated with popup, typically via aria-controls.
- **popup-listbox:** Popup exposes listbox semantics for static options.
- **option-roles:** Selectable items expose option semantics.
- **selected-state:** Selected option state is programmatically represented.
- **keyboard-arrow-down-opens:** ArrowDown opens popup and identifies/activates first option.
- **keyboard-enter-selects:** Enter accepts the active/focused option.
- **keyboard-escape-closes:** Escape closes popup.
- **axe-scan:** Run axe-core scoped to component root.
- **manual-only boundaries:** Screen reader quality, native-select preference, visual focus, and cross-browser AT behavior remain manual.

9. Applicability and reject-if policy

The manifest should declare when it is allowed to apply. This intentionally duplicates some Phase 2 safety logic. Phase 2 is the classifier; Phase 3 is the trust gate.

```
applicability.requires:
- single selected value
- static option collection
- trigger-controlled popup
- mapped selectable options
- no freeform text input

applicability.rejectIf:
- text input inside popup
- checkbox inside option subtree
- link inside option subtree
- action button inside option subtree
- multiple-selection state
- nested interactive controls inside options
- navigation behavior inside option subtree
```

10. Phase 3 output artifact

Phase 3 should write manifestLoadResult.json. This artifact proves that the rulebook was selected from the registry, found on disk, schema-valid, source-layered, and safe to hand to Phase 4.

```
{
"schemaVersion": "1.0.0",
"component": { "name": "SelectOnlyCombobox", "path": "..." },
"classification": {
"pattern": "react-select-only-combobox",
"status": "supported",
"confidence": 0.92
},
"manifest": {
"id": "react-select-only-combobox",
"version": "1.0.0",
"path": "specfirst/rules/react-select-only-combobox.v1.json",
```

```
"status": "loaded",
"sourceModel": "7-layer"
},
"sourceLayers": [
"wcag-outcomes",
"wai-aria-apg-pattern",
"aria-validity",
"accessible-name",
"automation",
"manual-review-boundary",
"specfirst-safety-policy"
],
"validation": {
"registryEntryFound": true,
"manifestFileFound": true,
"schemaValid": true,
"patternMatchesClassification": true,
"applicabilityValidated": true,
"hasManualReviewBoundary": true,
"hasReportBoundary": true
},
"nextPhase": {
"canContinue": true,
"loadedManifestPath": "specfirst/rules/react-select-only-combobox.v1.json"
}
}
```


11. Stopped-case behavior

If Phase 2 says canContinue=false, Phase 3 must not infer or guess a manifest. It should write a stopped manifestLoadResult with the reason and nextPhase.canContinue=false. This is especially important for mixed-interactive-popup, native-select/manual-review, unsupported, and ambiguous cases.

12. Validation checklist

- **classification.nextPhase.canContinue=true:** Hard precondition.
- **manifestId present:** Do not guess.
- **registry entry found:** Registry is the allowed manifest set.
- **manifest file found:** Local versioned rulebook must exist.
- **manifest JSON schema valid:** Catch malformed rules early.
- **sourceModel == 7-layer:** Prevents old flat manifests from silently loading.
- **all seven source sections exist:** Sections may be lightly implemented, but must exist.
- **requirements non-empty:** Cannot generate spec without requirements.
- **manualReviewRequired non-empty:** Prevents compliance overclaim.
- **reportBoundary forbids compliance claim:** Must forbid WCAG-compliant language.
- **applicability/rejectIf present:** Second safety gate.

13. What Phase 3 must not do

Phase 3 must not generate a component-specific spec, generate Playwright tests, run axe, call Bob, patch code, or claim the component is accessible. It only loads and validates the trusted rulebook. Phase 4 turns the manifest into a locked spec; Phase 5 turns that spec into tests.

14. Definition of done

```
Phase 3 is complete when:
[ ] Reads classification.json explicitly.
```

```
[ ] Stops if canContinue=false.
[ ] Loads registry.json.
[ ] Finds the manifest entry.
[ ] Loads the versioned local manifest file.
[ ] Validates manifest schema.
[ ] Validates sourceModel=7-layer.
[ ] Validates all seven source-layer sections exist.
[ ] Validates requirements, automation, manual review, safety policy, and
report boundary exist.
[ ] Copies manifestUsed.json into the run folder.
[ ] Writes manifestLoadResult.json.
[ ] Sets nextPhase.canContinue=true only when all gates pass.
[ ] Does not call Bob, generate tests, or make compliance claims.
```

15. Short implementation direction

Implement Phase 3 as a deterministic loader and validator. Start with a pure function: loadManifestForClassification(classification, registry) -> manifestLoadResult. Wrap it with a CLI after the pure function works. Use JSON schema validation for the manifest shape. Keep the only MVP-supported manifest as react-select-only-combobox.v1.json. Store both the loaded manifest snapshot and the load result beside the current run artifacts.